

INSTITUT INTERNATIONAL D'INGÉNIERIE DE L'EAU
ET DE L'ENVIRONNEMENT
2iE

Filière : Informatique, Intelligence Artificielle et Applications

RAPPORT DE PROJET – CLOUD COMPUTING (S8)

MicroScore

Plateforme de Diagnostic Médical
IA Native Cloud

Conception, déploiement et sécurisation d'une application de diagnostic
médical assistée par intelligence artificielle sur infrastructure AWS

Réalisé par :

OUEDRAOGO Freddy
SAWADOGO Imaane

Année académique :

2025 – 2026

Table des matières

1	Introduction	3
1.1	Contexte général	3
1.2	Objectifs du projet	3
1.3	Démarche méthodologique	3
2	Analyse fonctionnelle et besoins	5
2.1	Présentation générale de la solution	5
2.2	Fonctionnalités clés	5
2.2.1	Intelligence artificielle et diagnostic	5
2.2.2	Tableau de bord administrateur (Analytics)	5
2.2.3	Sécurité et performance	6
2.3	Acteurs et cas d'usage	6
3	Architecture technique	7
3.1	Stack technologique	7
3.1.1	Backend	7
3.1.2	Frontend	7
3.1.3	Infrastructure et DevOps	7
3.2	Architecture cloud sur AWS	7
3.2.1	Schéma de flux	8
3.2.2	Description des composants	8
4	Sécurité de l'application	9
4.1	Authentification OAuth 2.0	9
4.2	Contrôle d'accès basé sur les rôles (RBAC)	9
4.3	Protection contre les abus (Rate Limiting)	10
4.4	Sécurisation de l'infrastructure	10
5	Présentation de l'application	11
5.1	Parcours utilisateur	11
5.2	Tableau de bord administrateur	11
6	Modèle de Machine Learning	12

6.1	Choix du modèle	12
6.2	Caractéristiques du modèle	12
6.3	Pipeline d'entraînement	12
7	Déploiement et intégration continue	13
7.1	Conteneurisation	13
7.2	Installation locale	13
7.2.1	Prérequis	13
7.2.2	Lancement rapide	13
7.3	Structure du projet	13
7.4	Pipeline CI/CD	14
7.5	Déploiement final	14
8	Difficultés rencontrées et solutions apportées	15
8.1	Configuration de l'authentification OAuth en environnement Dockerisé	15
8.2	Arbitrage entre sécurité et coût de l'infrastructure	15
9	Conclusion et perspectives	16
9.1	Conclusion	16
9.2	Perspectives d'amélioration	16
	Références	17

1.1 Contexte général

L'accès rapide à un diagnostic médical préliminaire reste un défi majeur dans de nombreux contextes, notamment lorsque les structures de santé de premier recours sont éloignées ou surchargées. Les pathologies respiratoires et intestinales courantes (rhume, grippe, COVID-19, bronchite, pneumonie, gastro-entérite) partagent souvent des symptômes similaires, ce qui rend leur identification précoce difficile sans outil d'aide à la décision.

C'est dans cette optique que s'inscrit le projet **MicroScore**, une plateforme de diagnostic médical assisté par intelligence artificielle, déployée sur une infrastructure cloud native AWS. Ce projet a été réalisé dans le cadre du module de Cloud Computing de la filière Informatique, Intelligence Artificielle et Applications de l'Institut 2iE.

1.2 Objectifs du projet

L'objectif principal de ce projet est de concevoir, développer et déployer une application web complète permettant :

- d'évaluer un ensemble de symptômes saisis par un utilisateur et de proposer une prédiction de pathologie probable à l'aide d'un modèle de Machine Learning ;
- de sécuriser l'accès à l'application via une authentification OAuth 2.0 (Google) ;
- de fournir aux administrateurs un tableau de bord analytique permettant le suivi des diagnostics réalisés ;
- de déployer l'ensemble de la solution sur une architecture cloud robuste, scalable et économiquement optimisée sur AWS ;
- de mettre en place une chaîne d'intégration et de déploiement continu (CI/CD).

1.3 Démarche méthodologique

Le projet a été structuré en plusieurs phases : analyse des besoins et choix technologiques, entraînement et validation du modèle de classification, développement du backend et du frontend, conteneurisation des services, conception de l'architecture

cloud, puis déploiement et sécurisation sur AWS. Une démarche itérative a été adoptée, avec deux livrables intermédiaires permettant de valider successivement l'analyse des risques/coûts et l'implémentation technique.

Chapitre 2

Analyse fonctionnelle et besoins

2.1 Présentation générale de la solution

MicroScore est un système de diagnostic médical intelligent conçu pour assister les professionnels de santé dans l'identification rapide de maladies respiratoires et intestinales courantes. L'application combine un modèle d'intelligence artificielle performant avec une architecture cloud robuste et sécurisée sur AWS.

L'application est accessible à l'adresse suivante :

<http://microscore-alb-875257365.eu-west-1.elb.amazonaws.com/>

2.2 Fonctionnalités clés

2.2.1 Intelligence artificielle et diagnostic

- **Analyse des symptômes** : évaluation de 17 symptômes clés pour générer des prédictions.
- **Classification multi-maladies** : détection de 6 pathologies (Rhume, Grippe, COVID-19, Bronchite, Pneumonie, Gastro-entérite).
- **Score de confiance** : chaque diagnostic est accompagné d'un indice de certitude issu du modèle de Machine Learning.
- **Modèle performant** : utilisation d'un `RandomForestClassifier` atteignant une précision de **99,2 %**.

2.2.2 Tableau de bord administrateur (Analytics)

- **KPI en temps réel** : suivi du nombre total de diagnostics, de la confiance moyenne et des tendances hebdomadaires.
- **Visualisation de données** : graphiques dynamiques (Recharts) montrant la distribution des maladies et l'évolution quotidienne.
- **Historique complet** : consultation détaillée de toutes les prédictions passées, à des fins d'audit et d'analyse.

2.2.3 Sécurité et performance

- **Authentication OAuth 2.0** : connexion sécurisée via Google Auth.
- **Protection par limitation de débit (Rate Limiting)** : utilisation de SlowAPI pour prévenir le *model stealing* et les abus.
- **RBAC (Role-Based Access Control)** : accès restreint aux fonctionnalités administratives.
- **Base de données persistante** : stockage sécurisé sur PostgreSQL 16.

2.3 Acteurs et cas d'usage

Acteur	Cas d'usage principal
Utilisateur / Patient	Se connecter via Google, renseigner ses symptômes et obtenir une prédiction de diagnostic avec score de confiance.
Administrateur	Consulter le tableau de bord analytique, visualiser l'historique des diagnostics et superviser l'usage de la plateforme.
Système (Backend ML)	Recevoir les symptômes, exécuter l'inférence du modèle <code>RandomForestClassifier</code> et renvoyer la prédiction.

Tableau 2.1: Acteurs et cas d'usage de la plateforme MicroScore

Chapitre 3

Architecture technique

3.1 Stack technologique

3.1.1 Backend

- **Framework** : FastAPI (Python 3.11+)
- **Machine Learning & Data** : Scikit-learn, Joblib, Pandas
- **ORM & Base de données** : SQLAlchemy, Psycopg2, PostgreSQL 16
- **Sécurité** : PyJWT, Google Auth, SlowAPI
- **Intégration Cloud** : Boto3 (AWS S3)

3.1.2 Frontend

- **Framework** : React 18 (Vite)
- **Navigation** : React Router 6
- **Visualisation** : Recharts
- **Service** : Nginx (conteneurisé)

3.1.3 Infrastructure et DevOps

- **Cloud** : AWS (ALB, ECS Fargate, RDS, S3, ECR)
- **Conteneurisation** : Docker, Docker Compose
- **CI/CD** : GitHub Actions (Linting, Tests, Déploiement automatique vers ECS)

3.2 Architecture cloud sur AWS

Le projet propose deux stratégies de déploiement sur AWS, détaillées dans `infra/aws/architecture.md` :

1. **Variante Production** : sécurité maximale avec tâches ECS placées dans des sous-réseaux privés, associée à une NAT Gateway pour l'accès sortant.

2. **Variante Budget** : optimisation des coûts (suppression du coût de la NAT Gateway) en utilisant des sous-réseaux publics sécurisés par des Security Groups stricts.

3.2.1 Schéma de flux

La figure 3.1 présente le schéma de flux global de l'architecture déployée. Les requêtes des utilisateurs transitent par un *AWS Load Balancer* qui les redirige soit vers le frontend React (ECS), soit vers le backend FastAPI (ECS). Le backend communique ensuite avec la base de données RDS PostgreSQL, le bucket S3 contenant le modèle de Machine Learning, ainsi qu'avec le service Google OAuth pour l'authentification.

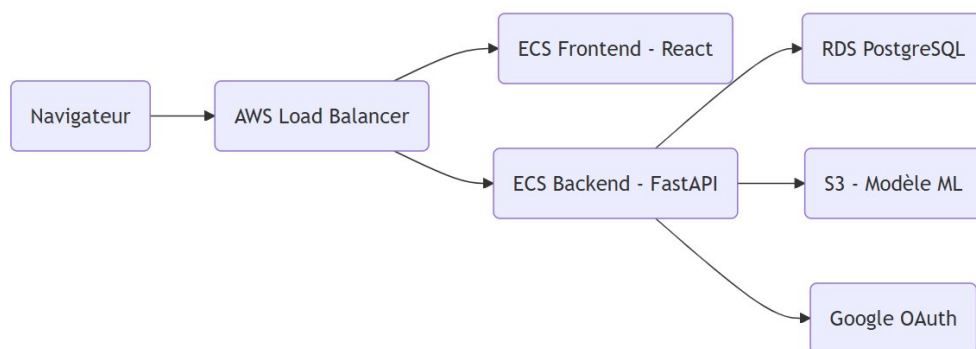


Figure 3.1: Schéma de flux de l'architecture cloud MicroScore

3.2.2 Description des composants

Composant	Rôle
AWS Load Balancer (ALB)	Point d'entrée unique, répartition du trafic vers les services frontend et backend.
ECS Frontend – React	Sert l'interface utilisateur statique compilée (build Vite), conteneurisée et exécutée sur ECS Fargate.
ECS Backend – FastAPI	Expose l'API REST, exécute la logique métier, l'authentification et l'inférence du modèle ML.
RDS PostgreSQL	Stocke de manière persistante les utilisateurs, les diagnostics et les historiques.
S3 – Modèle ML	Héberge le modèle <code>RandomForestClassifier</code> sérialisé (Joblib), chargé dynamiquement par le backend.
Google OAuth	Fournit l'authentification sécurisée des utilisateurs et administrateurs.

Tableau 3.1: Description des composants de l'architecture

Chapitre 4

Sécurité de l'application

4.1 Authentification OAuth 2.0

L'accès à l'application est entièrement sécurisé par une authentification déléguée via **Google OAuth 2.0**. La page de connexion (figure 4.1) ne propose qu'un seul point d'entrée : le bouton "*Continuer avec Google*", ce qui élimine la gestion de mots de passe côté application et réduit la surface d'attaque.

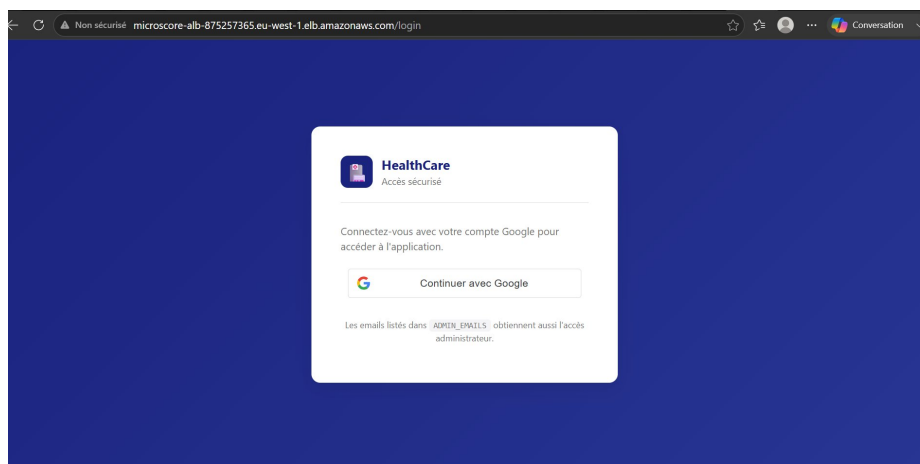


Figure 4.1: Page de connexion sécurisée de MicroScore

Les adresses e-mail listées dans la variable d'environnement **ADMIN_EMAILS** obtiennent automatiquement le rôle administrateur lors de leur première connexion, ce qui permet une gestion simple et centralisée des droits d'accès.

4.2 Contrôle d'accès basé sur les rôles (RBAC)

Un mécanisme de **RBAC** restreint l'accès aux fonctionnalités sensibles (tableau de bord analytique, historique complet des diagnostics) aux seuls comptes disposant du rôle administrateur. Les utilisateurs standards n'ont accès qu'au formulaire de diagnostic et à leurs propres résultats.

4.3 Protection contre les abus (Rate Limiting)

Afin de prévenir le *model stealing* (extraction non autorisée du modèle par requêtes massives) ainsi que les abus de service, l'API backend intègre **SlowAPI**, une bibliothèque de limitation de débit. Chaque endpoint sensible, en particulier celui de prédiction, est soumis à un quota de requêtes par intervalle de temps et par utilisateur/adresse IP.

4.4 Sécurisation de l'infrastructure

Au niveau infrastructure, la sécurité repose sur :

- des **Security Groups** strictement configurés, n'autorisant que les flux nécessaires entre les composants (ALB → ECS, ECS → RDS, ECS → S3) ;
- l'isolement des tâches ECS en sous-réseaux privés dans la variante Production ;
- le chiffrement des jetons d'authentification via **PyJWT** ;
- le stockage sécurisé et persistant des données sensibles dans **PostgreSQL 16** géré par RDS.

Chapitre 5

Présentation de l'application

5.1 Parcours utilisateur

Après authentification réussie via Google, l'utilisateur est redirigé vers le formulaire de diagnostic médical (figure 5.1). Ce formulaire lui permet de renseigner, de manière optionnelle, le nom du patient (utile pour l'historique médical), puis de sélectionner parmi les 17 symptômes proposés ceux qu'il présente (fièvre, toux, fatigue, difficulté respiratoire, etc.).

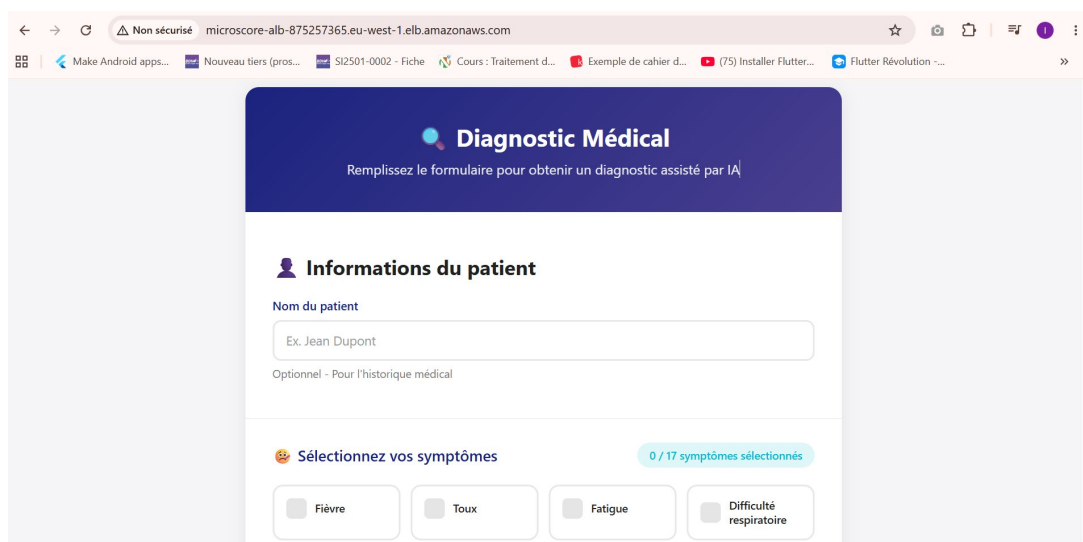


Figure 5.1: Formulaire de diagnostic médical assisté par IA

Une fois les symptômes sélectionnés et le formulaire soumis, la requête est transmise au backend FastAPI, qui charge le modèle `RandomForestClassifier` depuis le bucket S3 et retourne la pathologie prédite, accompagnée de son score de confiance.

5.2 Tableau de bord administrateur

Les comptes disposant du rôle administrateur accèdent à un tableau de bord présentant les indicateurs clés de performance (KPI) en temps réel : nombre total de diagnostics réalisés, confiance moyenne des prédictions, tendances hebdomadaires, ainsi que des graphiques de distribution des maladies générés avec Recharts. Cet espace permet également la consultation de l'historique complet des prédictions à des fins d'audit.

Chapitre 6

Modèle de Machine Learning

6.1 Choix du modèle

Le cœur prédictif de MicroScore repose sur un algorithme **RandomForestClassifier**, choisi pour sa robustesse face au bruit dans les données, sa capacité à gérer des variables catégorielles binaires (présence/absence de symptôme) et sa résistance au sur-apprentissage grâce à l'agrégation de multiples arbres de décision.

6.2 Caractéristiques du modèle

Caractéristique	Valeur
Algorithme	RandomForestClassifier (Scikit-learn)
Nombre de symptômes (features)	17
Nombre de classes (maladies)	6 (Rhume, Grippe, COVID-19, Bronchite, Pneumonie, Gastro-entérite)
Précision obtenue	99,2 %
Sérialisation	Joblib
Hébergement du modèle	AWS S3

Tableau 6.1: Caractéristiques du modèle de classification

6.3 Pipeline d'entraînement

Le modèle est entraîné hors ligne à partir d'un jeu de données de symptômes et de diagnostics associés, en utilisant les bibliothèques **Pandas** pour la préparation des données et **Scikit-learn** pour l'entraînement et la validation. Le modèle final est sérialisé via **Joblib**, puis téléversé sur un bucket **AWS S3**. Le backend FastAPI télécharge et charge dynamiquement ce modèle au démarrage du service grâce à la bibliothèque **Boto3**, ce qui permet de mettre à jour le modèle sans nécessiter de redéploiement complet de l'application.

Chapitre 7

Déploiement et intégration continue

7.1 Conteneurisation

L'ensemble des services de l'application (frontend, backend) est conteneurisé avec **Docker**. Le fichier `docker-compose.yml` orchestre ces services en environnement local, facilitant ainsi le développement et les tests avant déploiement.

7.2 Installation locale

7.2.1 Prérequis

- Docker & Docker Compose
- Compte Google Cloud (pour OAuth, optionnel en local)

7.2.2 Lancement rapide

```
1 git clone <url-du-depot>
2 cd projet-cloud-s8
3 cp .env.example .env
4 docker compose up --build
```

- Frontend : <http://localhost:5173>
- Backend API : <http://localhost:8000>
- Documentation Swagger : <http://localhost:8000/docs>

7.3 Structure du projet

```
1 |-- backend/           # API FastAPI et logique ML
2 |-- frontend/          # Application React et tableau de bord
3 |-- infra/aws/         # Templates et schemas d'architecture
4 |-- scripts/           # Scripts d'entraînement et DevOps
5 |-- docs/              # Documentation détaillée et rapports
6 |-- .github/workflows/ # Pipelines CI/CD
```

```
7 -- docker-compose.yml # Orchestration locale
```

7.4 Pipeline CI/CD

L'intégration et le déploiement continus sont assurés par **GitHub Actions**. À chaque mise à jour du code, le pipeline exécute automatiquement :

1. le *linting* du code (qualité et conventions) ;
2. l'exécution des tests automatisés ;
3. la construction des images Docker du frontend et du backend ;
4. le déploiement automatique vers les services **AWS ECS**.

Cette automatisation garantit une livraison continue, fiable et reproductible des évolutions de la plateforme.

7.5 Déploiement final

L'application est actuellement déployée et accessible à l'adresse :

<http://microscore-alb-875257365.eu-west-1.elb.amazonaws.com/>

Chapitre 8

Difficultés rencontrées et solutions apportées

8.1 Configuration de l'authentification OAuth en environnement Dockerisé

L'intégration de Google OAuth 2.0 dans un environnement conteneurisé a soulevé une difficulté liée à une incohérence entre l'URI de redirection configurée côté Google Cloud Console et celle réellement utilisée par l'application déployée derrière le Load Balancer AWS, accentuée par la présence d'identifiants de test (placeholders) dans la configuration initiale. La résolution a nécessité l'alignement précis des URI de redirection autorisées avec le nom de domaine public de l'ALB, ainsi qu'une gestion rigoureuse des variables d'environnement (`.env`) propres à chaque environnement (local et production).

8.2 Arbitrage entre sécurité et coût de l'infrastructure

Le choix entre les variantes Production (sous-réseaux privés avec NAT Gateway) et Budget (sous-réseaux publics avec Security Groups stricts) a constitué un arbitrage important, documenté dans l'analyse des risques et des coûts du projet. La variante Budget a été retenue pour la démonstration afin d'éliminer le coût récurrent de la NAT Gateway, tout en maintenant un niveau de sécurité acceptable grâce à des règles de pare-feu strictement définies.

Conclusion et perspectives

9.1 Conclusion

Le projet **MicroScore** a permis de mettre en application l'ensemble des compétences acquises en Cloud Computing : conception d'architecture distribuée, conteneurisation, sécurisation des accès, intégration d'un modèle de Machine Learning dans une chaîne de production, et mise en place d'une chaîne CI/CD complète. La plateforme finale offre une expérience utilisateur fluide, une sécurité renforcée par l'authentification déléguée et le RBAC, ainsi qu'une précision de prédiction élevée (99,2 %) grâce au modèle `RandomForestClassifier`.

9.2 Perspectives d'amélioration

Plusieurs axes d'amélioration peuvent être envisagés pour les développements futurs :

- l'ajout d'un système de journalisation et de monitoring centralisé (CloudWatch, alerting) ;
- l'extension du jeu de données et du nombre de pathologies détectées par le modèle ;
- la mise en place d'un cache (Redis) pour améliorer les temps de réponse de l'API ;
- le renforcement du chiffrement des données au repos et en transit (HTTPS via certificat ACM) ;
- l'industrialisation complète de l'infrastructure via un outil d'Infrastructure as Code (Terraform).

Références

- Rapport de Projet Final : https://github.com/SAWADOGO-Imaane/Projet_cloud_S8/blob/main/docs/livrable2/rapport-livrable2.md
- Analyse des Risques et Coûts : https://github.com/SAWADOGO-Imaane/Projet_cloud_S8/blob/main/docs/livrable1/rapport-livrable1.md
- Spécifications du Modèle ML : https://github.com/SAWADOGO-Imaane/Projet_cloud_S8/blob/main/docs/HEALTHCARE_MODEL.md